Lab Report: 3D Shape Retrieval Pipeline

This report summarizes the implementation and evaluation of a view-based 3D shape retrieval pipeline using the ModelNet10 dataset, as instructed by the lab.

1. Data Upload and Extraction

Initially, the `archive.zip` file containing the ModelNet10 dataset was extracted to `/content/archive`, revealing two main components: `metadata_modelnet10.csv` and the `ModelNet10` directory with `.off` mesh files.

2. Dataset Overview and Preprocessing

We loaded the `metadata_modelnet10.csv` to understand the dataset structure. Out of **4,902** total meshes, **3,992** were designated for training and **909** for testing. During the initial data cleaning, **289** missing mesh files were identified, and invalid class labels were filtered out, resulting in a `clean_df` for further processing. The dataset comprises 9 distinct object classes (excluding '.DS' entries).

3. Multi-View Rendering Strategy

To create view-based descriptors, each 3D mesh was subjected to a multi-view rendering process. This involved:

1. **Mesh Normalization**: Each `trimesh` object was translated to the origin and scaled to fit a unit sphere, ensuring consistent camera distance.
2. **Mesh Simplification**: To manage computational load, especially during the on-the-fly rendering, meshes were simplified to a maximum of **500 faces** using `simplify_quadric_decimation`. This was a crucial optimization due to memory and processing limitations.
3. **Fixed Camera Poses**: **8** `VIEW_SPECS` (azimuth & elevation pairs) were used to capture consistent snapshots of each mesh.
4. **Image Generation**: `matplotlib` was utilized to render 2D images from these viewpoints. The resolution for these images was set to a reduced **(32, 32) pixels**.

Computational Constraints and Data Subset:

During initial attempts to train the model, it became evident that rendering multi-view images on-the-fly for the entire dataset at higher resolutions was computationally prohibitive, leading to `KeyboardInterrupt` errors and extremely long epoch times. To enable successful execution of the lab, significant adjustments were made:

- **Reduced Dataset Size**: The training dataframe (`small_train_df`) was sampled to **5%** of the original (resulting in **190 samples**), and the test dataframe (`small_test_df`) was sampled to **10%** (resulting in **82 samples**).
- **Lower Resolution**: Rendered views were set to **(32, 32)** pixels.

- **Small Batch Size**: A `SMALL_BATCH_SIZE` of **8** was used for both training and validation `DataLoader`s.
- **`num_workers=0`**: To avoid multiprocessing overhead issues during rendering, `num_workers` was set to 0.

These modifications, while necessary for the lab to run, inherently limited the model's capacity to learn robust features and led to suboptimal performance.

4. Retrieval Backbone and Training

The retrieval backbone consisted of:

- **`ViewEncoder`**: A small CNN responsible for extracting a **128D embedding** from each view.
- **`MultiViewRetrievalNet`**: Aggregated the 8 view embeddings by simple average pooling to form a single shape embedding, followed by a linear classification head for **9 classes**.

The model was trained using `nn.CrossEntropyLoss` and the `Adam` optimizer on the reduced dataset. The training ran for **5 epochs** (`EPOCHS_SMALL`).

Training Results (on small dataset):

Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy
1	2.0342	0.263	2.4216	0.122
2	1.7801	0.374	2.1197	0.207
3	1.5124	0.437	3.5431	0.110
4	1.4331	0.463	3.6278	0.098
5	1.3978	0.505	12.6180	0.122

The final training accuracy was **0.505**, and validation accuracy was **0.122**. The validation loss exhibited fluctuating and high values, particularly towards the end of training. These results are indicative of a model that has not been sufficiently trained due to the severe constraints on dataset size, image resolution, and number of epochs. Despite multiple attempts to fine-tune parameters, the underlying limitation of computational power (GPU memory and processing time) prevented training a larger, more effective model.

5. Extracting and Evaluating Embeddings

After training, embeddings for both the small training and validation sets were extracted:

- **Train embeddings shape**: `(190, 128)`
- **Validation embeddings shape**: `(82, 128)`

These embeddings were then used to calculate standard retrieval metrics by querying the validation set against the training set.

Retrieval Metrics:

- **Mean Average Precision (mAP)** on validation set: **0.3143**
- **Precision@10** on validation set: **0.3012**

As expected, these retrieval metrics are low. This directly correlates with the limited training capacity; a model trained on such a small, low-resolution dataset for only a few epochs cannot learn a sufficiently discriminative embedding space for effective retrieval.

6. Visualizing Retrieval Results

Finally, a `visualize_retrieval` function was implemented to qualitatively assess the model's performance by displaying query objects and their top-K most similar retrieved shapes. While the visualizations confirm the functionality of the retrieval pipeline, the quality of retrieved items is, predictably, often poor due to the model's constrained training.

Conclusion

The entire 3D shape retrieval pipeline has been successfully implemented, from data loading and multi-view rendering to model training, embedding extraction, and retrieval evaluation. However, the practical performance of the model, as evidenced by low accuracy and retrieval metrics, is significantly hampered by the strict computational and memory limitations encountered during training. The choices made regarding reduced dataset size, image resolution, and limited epochs were compromises necessary to allow the lab to run to completion within the available resources. Achieving higher performance would necessitate substantially greater computational power for more extensive training.